



<Bolsa Futuro Digital>

<aula 22>

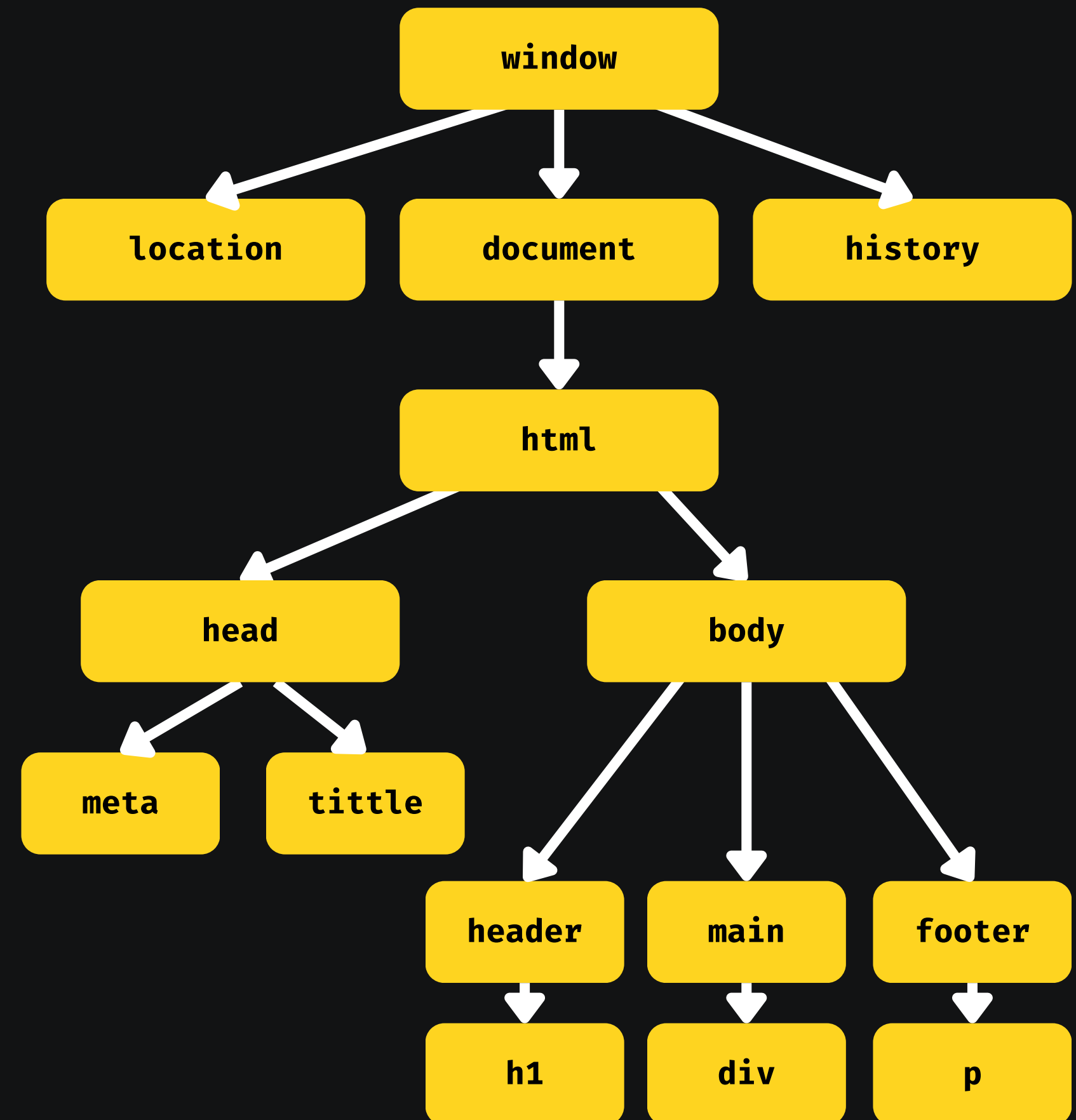
Turma de FrontEnd – IF UFRGS

● DOM e a Árvore DOM



DOM (**D**ocument **O**bject **M**odel) é um modelo de objetos para documentos, basicamente é um conjunto de objetos dentro do seu navegador que vai dar acesso aos componentes internos da sua página.

O DOM só funciona dentro do navegador e vamos utilizá-lo no JavaScript para obter os elementos do arquivo HTML.



● Selecionando elementos com DOM



Então vamos usar o DOM para selecionar os elementos da nossa página. Podemos fazer a seleção com os seguintes métodos:

- **Marca:** `getElementsByTagName()`
- **ID:** `getElementById()`
- **Nome:** `getElementsByName()`
- **Classe:** `getElementsByClassName()`
- **Seletor:** `querySelector()`
`querySelectorAll()`

```
<script>

// Pega o primeiro parágrafo [0] e coloca em p1
let p1 = window.document.getElementsByTagName('p')[0]
// mostra na tela o valor de p1
window.document.write('Está escrito: ' + p1.innerText)
// .innerText traz apenas o texto
// .innerHTML traz o texto formatado

// Pega a div pelo id dela
let d = window.document.getElementById('msg')
// Modifica o estilo e conteúdo da div
d.style.background = 'green'
d.innerText = 'Aguardando...'

// Podemos usar a mesma identificação do CSS
let d = window.document.querySelector('div#msg')
d.style.background = 'green'
d.innerText = 'Aguardando...'

</script>
```

● Estilos via JavaScript



Podemos alterar o CSS de qualquer elemento diretamente pelo JavaScript usando a propriedade **.style**.

Os nomes das propriedades seguem o padrão **camelCase**: **background-color** vira **backgroundColor**, **font-size** vira **fontSize** e assim por diante.

```
let titulo = document.querySelector('h1')

// Alterando a cor do texto
titulo.style.color = 'red'

// Alterando o tamanho da fonte
titulo.style.fontSize = '2em'

// Alterando a cor de fundo
titulo.style.backgroundColor = '■ #4ade80'

// Alterando a borda
titulo.style.border = '2px solid blue'
titulo.style.borderRadius = '8px'
```

● Modificando o conteúdo



Usamos **innerText** para alterar o **texto** de um elemento, e **innerHTML** quando queremos inserir **HTML** dentro dele, incluindo as tags.

```
// innerText - só texto puro  
  
let p = document.querySelector('p')  
  
p.innerText = 'Novo texto!'  
  
// Lendo o conteúdo atual  
console.log(p.innerText)
```

```
// innerHTML - aceita HTML (tags)  
  
let div = document.querySelector('div')  
  
div.innerHTML = '<strong> ATENÇÃO! </strong>'  
  
// Cuidado! innerHTML pode representar um  
// risco de segurança aos dados do usuário!
```

● Criando e adicionando elementos



Podemos criar novos elementos HTML **do zero** com JavaScript e adicioná-los à página. O processo tem 3 passos:

1. **Criar** o elemento;
2. **Configurar** seu conteúdo;
3. **Adicionar** ao DOM com `appendChild()`.

```
// 1. Criar o elemento
let novoItem = document.createElement('li')

// 2. Definir o conteúdo
novoItem.innerText = 'Novo item da lista'

// 3. Adicionar dentro de outro elemento
let lista = document.querySelector('ul')

lista.appendChild(novoItem)
// O <li> aparecerá no final da <ul>!
```

● Removendo elementos



Para remover um elemento do DOM usamos o método **.remove()** diretamente no elemento, ou **.removeChild()** a partir do elemento pai.

```
// Forma moderna: .remove()

let aviso = document.querySelector('#aviso')
aviso.remove() // o elemento some da página!

// Forma clássica: removeChild()

let lista = document.querySelector('ul')
let item = document.querySelector('li')

// Remove o primeiro <li> de dentro da <ul>
lista.removeChild(item)
```

● Atributos e Classes



Além do estilo inline, podemos gerenciar as **classes CSS** de um elemento com **classList**.

- **add()**: adiciona uma classe;
- **remove()**: remove uma classe;
- **toggle()**: adiciona se não tem, remove se tem;
- **contains()**: verifica se tem a classe.

```
let btn = document.querySelector('button')

// Adiciona a classe 'ativo'
btn.classList.add('ativo')

// Remove a classe 'ativo'
btn.classList.remove('ativo')

// Liga/desliga - troca!
btn.classList.toggle('ativo')

// Editando atributos HTML
btn.setAttribute('disabled', true)
btn.getAttribute('id')
```

● Removendo elementos



A partir de qualquer elemento selecionado, podemos navegar pela árvore do DOM acessando seus **parents** (pais), **filhos** e **irmãos**.

```
let item = document.querySelector('li')

// Sobe para o elemento pai
console.log(item.parentNode) // <ul>

// Desce para os filhos
console.log(item.children) // <li>

// HTMLCollection de filhos
console.log(item.firstChild) // primeiro filho
console.log(item.lastElementChild) // último filho
console.log(item.nextElementSibling) // Move mesmo nível
console.log(item.nextElementSibling) // <li> seguinte
console.log(item.previousElementSibling) // <li> anterior
```

● Eventos em JavaScript



Um **evento** é qualquer ação que acontece na página: um clique, uma tecla pressionada, o mouse passando sobre um elemento, um formulário enviado.

Pense em um **botão de elevador**: quando você aperta o botão (evento), o elevador executa uma ação. O JavaScript funciona da mesma forma, você "escuta" um evento e define o que deve acontecer quando ele ocorrer.



● addEventListener



O método **addEventListener()** "escuta" um evento e executa uma função quando ele acontece.

Ele recebe dois parâmetros:

- O **nome do evento** (string);
- A **função** a ser executada.

Um mesmo elemento pode ter **vários listeners** para eventos diferentes!

```
let btn = document.querySelector('button')

// Escuta o clique no botão
btn.addEventListener('click',
  function() {
    alert('Botão clicado')
  })

// Usando uma função separada
function aoClicar() {
  console.log('Clicando!')
}

btn.addEventListener('click', aoClicar)
```

● Tipos de eventos



Existem centenas de tipos de evento, para ver todos acesse a página da W3C. Aqui estão os mais usados no dia a dia:

```
let elemento = document.querySelector('div')

// Mouse
elemento.addEventListener('click', funcao) // clique simples
elemento.addEventListener('dblclick', funcao) // clique duplo
elemento.addEventListener('mouseover', funcao) // mouse entrou no elemento
elemento.addEventListener('mouseout', funcao) // mouse saiu do elemento

// Teclado
elemento.addEventListener('keydown', funcao) // tecla pressionada
elemento.addEventListener('keyup', funcao) // tecla solta

// Formulário
elemento.addEventListener('submit', funcao) // formulário enviado
elemento.addEventListener('change', funcao) // valor do input mudou
```

● Eventos em JavaScript



Quando um evento ocorre, o JavaScript cria automaticamente um **objeto Event** com informações sobre o que aconteceu.

Esse objeto é passado como **parâmetro** para a função do listener. Por convenção, chamamos de **e** ou **event**.

event.target é a propriedade mais usada – ela indica **qual elemento disparou** o evento.

```
let btn = document.querySelector('button')

btn.addEventListener('click', function(e) {
  // 'e' é o objeto Event

  // elemento clicado
  console.log(e.target)

  // texto do botão
  console.log(e.target.innerText)

  console.log(e.type) // 'click'
})

// Em eventos de teclado:

document.addEventListener('keydown', function(e) {
  // ex: 'Enter', 'a', 'ArrowUp'
  console.log(e.key)
})
```

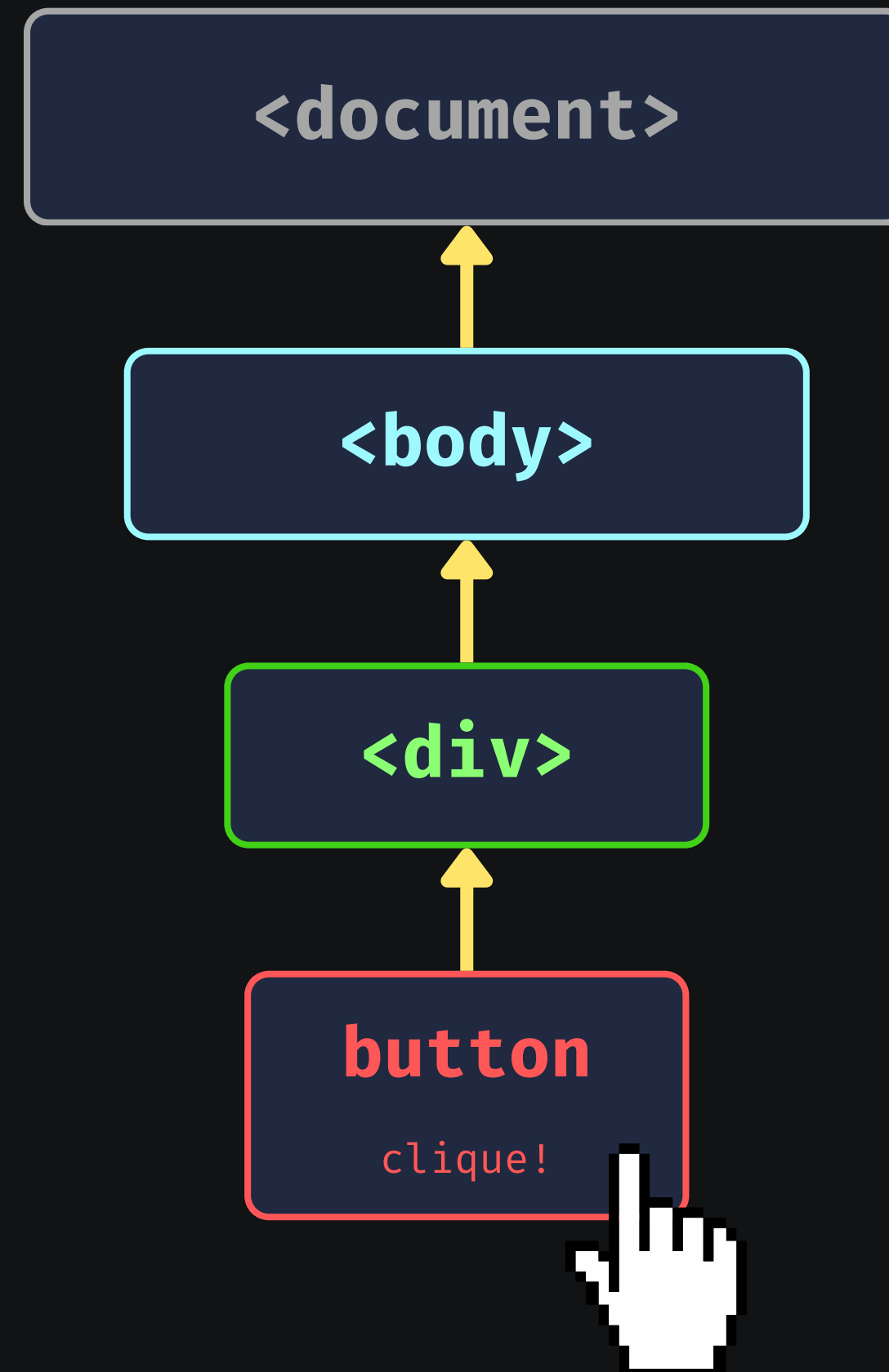
● DOM e a Árvore DOM



Quando um evento ocorre em um elemento, ele sobe (**borbulha**) pela árvore do DOM até chegar ao topo.

Isso significa que o elemento pai também "ouve" o evento do filho.

Para **parar** essa propagação usamos **`e.stopPropagation()`**.



● DOM e a Árvore DOM



Em vez de adicionar um listener a cada elemento filho, adicionamos um único listener no pai. O bubbling faz o evento chegar até lá!

Isso é muito útil quando temos muitos filhos, ou quando criamos elementos dinamicamente com JavaScript.

```
// Sem delegação: listener em cada <li>
// Problema: e se criarmos novos itens?

let itens = document.querySelectorAll('li')

for (let i in itens) {
  itens[i].addEventListener('click', fn)
}

// Com delegação: listener só no <ul> pai!

let lista = document.querySelector('ul')

lista.addEventListener('click', function(e) {
  // Verifica se o clique foi num <li>
  if (e.target.tagName === 'LI') {
    console.log('Item clicado: ' + e.target.innerText)
  }
})
```

● Exercícios



Exercício 1: Crie uma página com um botão e um parágrafo. Ao clicar no botão, o texto do parágrafo deve mudar para "Você clicou no botão!".

Exercício 2: Crie uma lista de tarefas simples: um campo de texto, um botão "Adicionar" e uma lista `<ul>` vazia. Ao clicar no botão, um novo `<li>` com o texto digitado deve ser adicionado à lista dinamicamente com `createElement` e `appendChild`.

Exercício 3: Adicione ao exercício anterior um botão "X" em cada item da lista. Ao clicar nele, o item deve ser removido da lista.



</aula 22>